4 Experiments and Results

This chapter reports the experiments on the GazeCapture data set [1] and on the personal recording sessions. It first describes the model, the data and the evaluation protocol (Section 4.1). It then analyses the training hyperparameters (Section 4.2), the spatial distribution of the prediction error and the loss functions (Section 4.3), and the personalization sessions (Section 4.4). Section 4.8 discusses what these results mean for the rest of the thesis.

All values were re-checked against the experiment logs. During this check an error in the evaluation code was found and corrected (Section 4.1.3). All model errors in this chapter are reported with the corrected metric.

4.1 Experimental Setup

4.1.1 Model

- **Backbone:** ResNet-18 [2], pretrained on ImageNet (`ResNet18_Weights.DEFAULT`).
- **Regression head:** four fully connected layers $512 \to 512 \to 256 \to 128 \to 2$ with ReLU, dropout $p = 0.2$ after the second layer, and an output activation (default: Sigmoid).
- **Optimizer:** AdamW, weight decay 10^{-5}.
- **Default loss:** L1 loss (`nn.L1Loss`), see Section 4.3.3.
- **Early stopping:** patience of three epochs, maximum 500 epochs.

4.1.2 Data and Splits

GazeCapture contains 347 085 labelled frames of 262 subjects in the version used here. Two splits are used:

- `norm_subject`: subject-disjoint split (no person appears in both training and test data);
- `norm_random`: frame-level split, the same person can appear in training and test data.

For the hyperparameter study, 1000 training and 200 test frames are drawn *at random* from the split files (fixed seed 0, `--sample-seed 0`). For `norm_subject` the training subset contains 179 subjects and the test subset 45 subjects, with no overlap. (Earlier runs used the *first* rows of the sorted split files, i.e. only five training subjects and one test subject; these runs are no longer used for the tables.)

4.1.3 Evaluation Metric

All gaze targets are normalized to the unit square $[0,1]^2$ (Section 4.1.5). The error is the root mean squared *Euclidean* distance between prediction $\hat{\boldsymbol{p}}_i$ and target $\boldsymbol{p}_i$, expressed as a percentage of the screen diagonal $\sqrt{2}$:

$$E_{\text{diag}} \;=\; 100 \cdot \frac{\sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\hat{\boldsymbol{p}}_i - \boldsymbol{p}_i\|_2^2}}{\sqrt{2}} \quad [\%]. \tag{4.1}$$

A value of 0 % is a perfect prediction, and 100 % is an error as large as the full screen diagonal.

Correction of the evaluation code. The previous evaluation function computed $\sqrt{\text{MSE}}$ with `sklearn.metrics.mean_squared_error`. For two-dimensional targets this function averages over the x and the y coordinate, so it returns the per-coordinate RMSE, which is the Euclidean RMSE divided by $\sqrt{2}$. Divided once more by $\sqrt{2}$, all *model* errors were exactly a factor $\sqrt{2}$ too small. The *baselines* were always computed with Equation (4.1) and were correct. The code was fixed (`metrics.py`, `model.py`, `model_train.py`, all notebooks). All hyperparameter results in Section 4.2 come from a new run with the corrected code. For older logs (personalization sessions, Table 4.13) the factor is exact, so their model errors were multiplied by $\sqrt{2}$. Example: a logged value of 20.08 % becomes $\sqrt{2} \cdot 20.08 = 28.40$ %.

4.1.4 Protocol

The hyperparameter study is run with the command

```
python exp.py --experiment all --sample-seed 0 --val-size 200.
```

One axis (for example the batch size) is changed, while all other values stay at the default configuration (Table 4.1). Each configuration is trained five times. Early stopping uses a subject-disjoint validation set of 200 frames, and the reported value is the *test* error at the epoch with the lowest validation error, so the test set is not used for model selection. The tables report the mean test error, the two-sided 98 % confidence interval (CI) of the mean (Student-t, four degrees of freedom) and the mean run time (defined below).

What "Run time" measures. The column *Run time* is the wall-clock duration of *one complete training run*, averaged over the five runs. A run starts with the evaluation of the untrained model (epoch 0) and ends when early stopping stops the training; it includes all training epochs and the evaluation on the training, validation and test data after every epoch. It is *not* the time per epoch, and it does *not* include loading the images or computing the cached backbone features, which happens once before the run. Because early stopping ends the runs after a different number of epochs, the time also reflects how long a configuration keeps improving.

Table 4.1: Default configuration of `exp.py`. Each sweep changes exactly one of these values.

Parameter	Value	Parameter	Value
Split	`norm_subject`	Trainable layers	FC only
Train / test size	1000 / 200[a]	Output activation	Sigmoid
Batch size	32	Loss	L1 (`standard`)
Learning rate	10^{-4}	Max. epochs / patience	500 / 3
Weight decay	10^{-5}	Validation set	200 frames[b]
Repeats	5		

[a] drawn at random, seed 0. [b] from one subject that is not in the training or test subset.

Reading the tables. In every hyperparameter table the selected option is marked with †. The column "p vs. †" gives the p-value of a two-sided Mann–Whitney U test between the five runs of that option and the five runs of the selected one. A rank test is used because five runs are too few to check normality and several options contain single outlier runs (e.g. one ReLU run at 46.98 %). With $5 + 5$ runs the smallest possible p-value is 0.008; it means that *every* run of one option is better than *every* run of the other. A mean is printed in **bold** only if the selected option is *significantly better* than every other option of the same table ($p < 0.05$ and lower mean).

Three properties of this protocol must be kept in mind:

1. **Same data in all repeats.** The seed fixes the train, test and validation subsets, so the five repeats differ only in weight initialization, data order and dropout. The CI therefore measures training randomness, not the variation between different data subsets.
2. **Small validation set.** The 200 validation frames come from a single subject. Early stopping is therefore noisy, but the test set is never used for model selection.
3. **Training time.** For FC-only runs the head is trained on cached ResNet features (512 numbers per image instead of the image). One run therefore takes only about one second, and the differences between these times are not relevant. The times of FC + Layer 4 and Full ResNet contain the forward and backward pass through the

backbone in every epoch and are therefore much longer; they cannot be compared directly with the FC-only times.

4.1.5 Target Normalization

The raw gaze targets are given in device-dependent units, so their range depends on the screen size and the orientation of the device. The targets are therefore normalized with the screen width W and height H:

$$x_{\text{norm}} = \frac{x}{W}, \qquad y_{\text{norm}} = \frac{y}{H}. \tag{4.2}$$

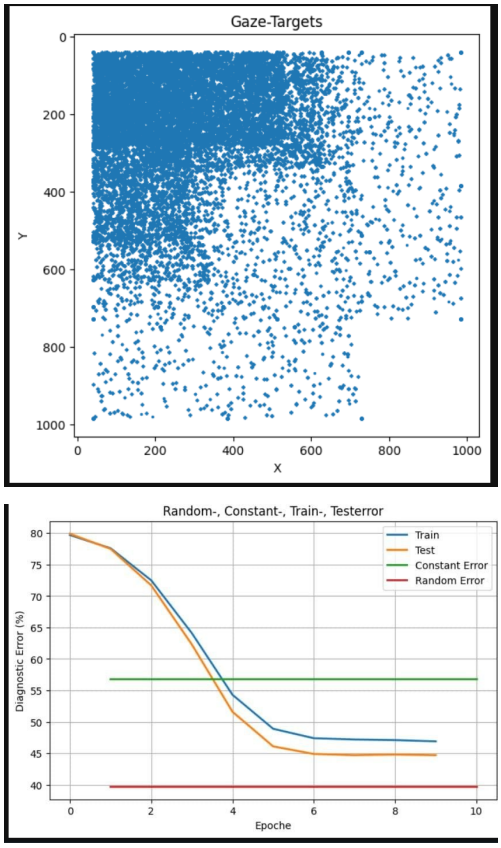
Figure 4.1 shows the effect. Without normalization the error values are not comparable between devices (left column). After normalization all targets lie in $[0, 1]^2$ and Equation (4.1) is well-defined (right column). All further experiments use normalized targets.

Why does the curve start at the constant and not at the random baseline? In Figure 4.1b the curves start at 26.77 % at epoch 0, i.e. *before* the first training step. One could expect the untrained model to behave like the random baseline (about 40 %). This expectation does not hold, for the following reason.

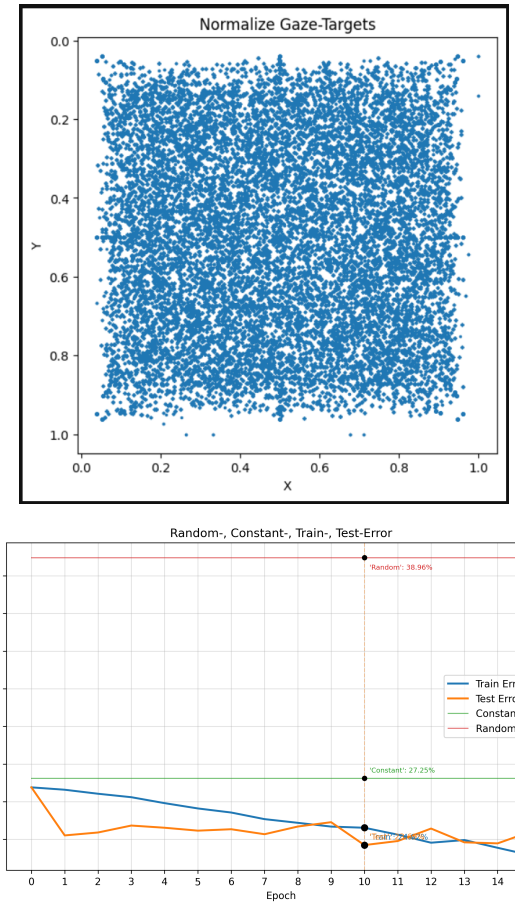
The random baseline guesses a *different*, uniformly distributed point for every image, spread over the whole screen. An untrained network does not do this. Its weights are initialized with small random values, so the last linear layer produces values close to 0 for every image, and the Sigmoid maps them to $\sigma(0) = 0.5$. The untrained model therefore predicts almost the *same* point, the screen centre (0.5, 0.5), for all images. This is exactly what the constant baseline does. Hence the correct expectation is:

- epoch 0 $\approx$ constant baseline of the *same* test data (a "always look at the centre" guess);
- training must bring the curve *below* this value; only then has the model learned something from the images;
- the random baseline is an upper reference for "pure guessing"; a model should never be above it.

The measured values confirm this: on the 200 test images of this run the constant baseline is 26.76 % (training mean) or 26.71 % (screen centre), which equals the start value of 26.77 %. The horizontal line in the old figure (27.25 %) was computed on the *full* data set and therefore lies slightly higher; this made the curve seem to start "below the baseline". The plotting code was corrected: the baselines are now computed on the same train/test subset as the curves (`metrics.subset_baselines`; `Compare_notebook.ipynb`, Section 4, which also re-creates this figure).



(a) Raw targets. Bottom: learning curve of an early run on raw coordinates; the values are not percentages of a common screen diagonal and cannot be compared with the other figures.



(b) Normalized targets. Bottom: learning curve on `norm_random` (1000/200) with the metric of Equation (4.1). Epoch 0 is the untrained model. The horizontal lines are baselines of the *full* data set; see text.

Figure 4.1: Raw versus normalized gaze targets.

4.1.6 Baselines

Two reference predictors are used. The *constant baseline* predicts the mean target of the training data for every test image. The *random baseline* draws predictions uniformly from $[0, 1]^2$. A model is only useful if it is clearly better than the constant baseline.

Table 4.2: Baseline errors E_{diag} in %. Top: full data set (`exp.py`, bootstrap of 30 samples with 2000 frames, mean and 98 % CI). Bottom: the same random train/test subsets (seed 0) that are used for model training (no CI, one fixed subset).

Data	Constant baseline	Random baseline
Full data set (347 085 frames)	27.28 [27.15, 27.40]	39.82 [39.60, 40.04]
`norm_subject`, 1000/200	28.46	40.36
`norm_subject`, 10 000/2000	27.62	39.96
`norm_random`, 1000/200	27.66	39.83

The baselines of the earlier draft (30.99 % and 42.70 %) came from an older notebook run that is not part of the logs; they are replaced by the logged values in Table 4.2. The lower part of the table is the correct reference for all model results in Section 4.2.

4.2 Training Configuration

All results in this section come from one run (`exp_results_all.json`; `exp.py`, corrected metric, random subsets with seed 0, validation-based early stopping, five repeats per configuration). The same-subset constant baseline of the default configuration is 28.46 %.

4.2.1 Batch Size

Table 4.3: Batch size. Mean test error over five runs with 98 % CI and mean duration of one complete training run (FC-only head on cached features, see Section 4.1.4).

Batch size	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)
32†	28.39	[28.21, 28.58]	—	0.019
64	28.35	[28.29, 28.41]	0.69	0.012
128	28.38	[28.30, 28.47]	0.84	0.009

Table 4.3 shows that the three batch sizes give the same test error: the means differ by at most 0.05 percentage points, the confidence intervals overlap, and the rank test finds no difference ($p \geq 0.69$). The training times are below one second per run, because only the small head is trained on cached features; the differences in time are therefore not relevant.

A batch size of 32 is kept as default. The experiment shows no disadvantage of this value, and there are two practical arguments for a small batch in the personalization setting: (i) a personal session contains only a few hundred training images, so a small batch gives more parameter updates per epoch, and (ii) the gradient noise of small batches is commonly associated with a mild regularizing effect. The data of this experiment do *not* prove an advantage of 32 over 64 or 128.

4.2.2 Learning Rate

Table 4.4: Learning rate. Mean test error over five runs with 98 % CI and mean duration of one complete training run.

Learning rate	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)
10^{-3}	28.43	[28.33, 28.54]	0.15	0.018
$10^{-4\dagger}$	28.37	[28.32, 28.42]	—	0.020
10^{-5}	28.33	[28.28, 28.38]	0.056	0.022
10^{-6}	28.37	[28.21, 28.52]	0.42	0.055

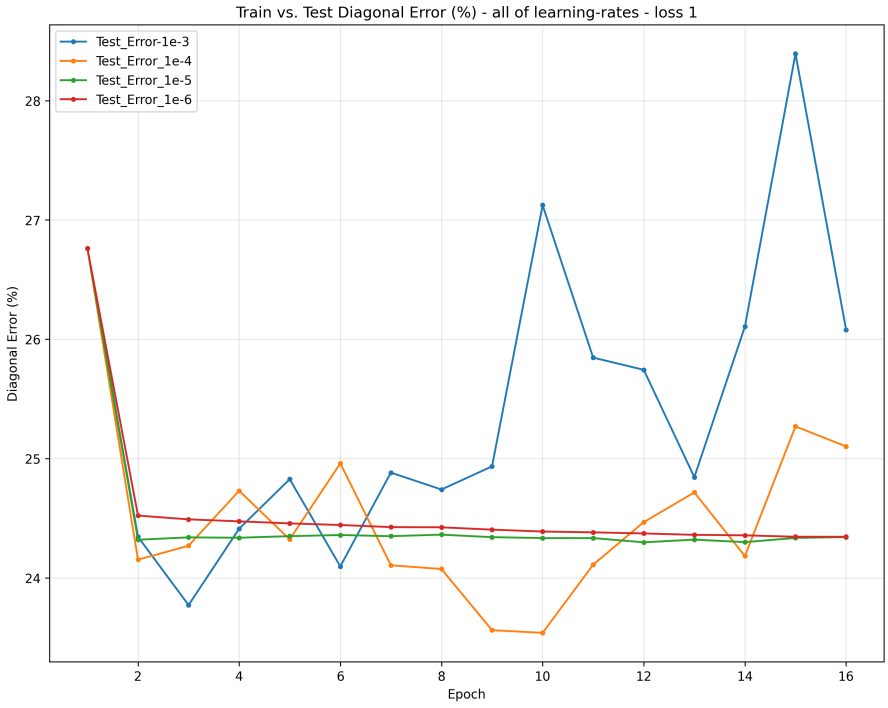


Figure 4.2: Test error curves of single training runs with four learning rates (`norm_random`, 1000/200, 15 epochs without early stopping, metric of Equation (4.1)). The first point (epoch 1 on the axis) is the untrained model.

Table 4.4 shows no significant difference between the four learning rates in the five-run sweep: all means lie between 28.33 % and 28.43 %, the confidence intervals overlap, and no

rank test is significant ($p \geq 0.056$). The final error alone therefore cannot justify the choice. The choice of $\eta = 10^{-4}$ is based on the *convergence behaviour* in Figure 4.2:

- $\boldsymbol{\eta = 10^{-3}}$: the steps are too large. After a fast first decrease the test error oscillates strongly and grows again (peaks of about 27.1 % and 28.4 %); the run does not converge to a stable minimum.
- $\boldsymbol{\eta = 10^{-4}}$: reaches the lowest test error of all curves (about 23.6 % around epoch 10). The curve still fluctuates, but much less than with 10^{-3}, and early stopping can keep the best state.
- $\boldsymbol{\eta = 10^{-5}}$ **and** $\boldsymbol{10^{-6}}$: the curves are very smooth and converge monotonically, but they stay on a plateau of about 24.3 %. The step size is too small to leave this region within the training budget; the result is stable but clearly not the best reachable one.

The training time is *not* used as a criterion here: it is below one second for all learning rates (cached features), and the decision depends only on convergence behaviour and the reachable error. Note that Figure 4.2 shows single runs; the five-run sweep in Table 4.4 does not confirm a significant difference.

4.2.3 Trainable Layers

Three fine-tuning depths are compared (Table 4.5): only the regression head (FC only), the head plus the last residual stage (`layer4`), and the complete network.

Table 4.5: Trainable parameters of the three fine-tuning configurations (ResNet-18 with regression head, 11 603 650 parameters in total).

	FC only	FC + Layer 4	Full ResNet
Trainable parameters	427 138	8 820 866	11 603 650
Share of all parameters	3.68 %	76.02 %	100.00 %
Backbone features cacheable	yes	no	no

Table 4.6: Trainable layers. Mean test error over five runs with 98 % CI and mean duration of one complete training run. The FC-only time excludes the backbone forward pass (cached features).

Trainable layers	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)
FC only†	28.36	[28.23, 28.49]	—	0.015
FC + Layer 4	29.07	[26.58, 31.55]	0.22	9.160
Full ResNet	29.39	[26.99, 31.78]	0.31	15.366

FC only has the lowest mean error and by far the narrowest interval (Table 4.6). Training more layers does not help; it makes the result *less stable*: one of the five FC + Layer 4 runs (31.72 %) and two of the five full fine-tuning runs (30.70 % and 31.18 %) end clearly above

the constant baseline (28.46 %), while all FC-only runs stay between 28.28 % and 28.47 %. With only five runs the differences are not significant ($p = 0.22$ and 0.31), so the FC-only value is not printed in bold. The training time grows strongly with the number of trainable parameters: 9.2 min per run for FC + Layer 4 and 15.4 min for the full network.

FC only is selected for the personalization experiments for three reasons:

1. **No loss in accuracy, more stability:** in this experiment, training more layers does not reduce the error but increases the spread between runs.
2. **Robustness with few data:** a personal session has only a few hundred images. Updating millions of backbone parameters with so few samples risks overfitting and distorts the pretrained features; Kumar et al. [3] show that full fine-tuning can damage pretrained representations and generalize worse than training only the head.
3. **Cost:** only 3.68 % of the parameters are trained and the backbone features can be computed once and reused. This makes personalization fast and memory-efficient.

4.2.4 Output Activation

The output activation is applied after the last linear layer that produces (x, y). Sigmoid maps the output to $(0, 1)$, which matches the normalized coordinate range. The Gaussian activation $g(z) = \exp(-z^2/2\sigma^2)$ with $\sigma = 1$ also maps to $(0, 1]$ but is not monotonic. ReLU only removes negative values, and "None" leaves the linear output unchanged.

Table 4.7: Output activation. Mean test error over five runs with 98 % CI and mean duration of one complete training run.

Activation	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)
Sigmoid†	**28.36**	[28.29, 28.43]	—	0.022
Gaussian	31.29	[28.44, 34.14]	0.008	0.019
ReLU	32.90	[19.69, 46.10]	0.008	0.028
None	29.39	[28.42, 30.36]	0.008	0.032

Sigmoid is the only clear winner of the hyperparameter study (Table 4.7): it has the lowest mean error and the narrowest interval, and *every* Sigmoid run is better than *every* run of the three other activations ($p = 0.008$ in all three comparisons). The value is therefore printed in bold. The other activations are also less stable: one ReLU run ends at 46.98 %, which is worse than the random baseline. A plausible reason is that ReLU outputs can become zero for all inputs ("dead" outputs), from which the gradient cannot recover. Without a bounding activation (None, ReLU) the network can also predict points outside the screen. Sigmoid is selected because it bounds the output to the valid screen range and gave the lowest and most stable error.

4.2.5 Data Set Size

Table 4.8: Data set size (train/test). Mean test error over five runs with 98 % CI, mean duration of one complete training run and the constant baseline of the respective test subset.

Train / test	Mean test error (%)	98 % CI (%)	p vs. †	Run time (min)	Const. baseline (%)
10 000 / 2000	27.64	[27.55, 27.73]	0.008	0.167	27.62
1000 / 200†	28.39	[28.33, 28.44]	—	0.018	28.46

The larger configuration has a lower mean error (Table 4.8). However, the two configurations are evaluated on *different* test sets, and each mean is practically equal to the constant baseline of its own test set (27.64 % vs. 27.62 % and 28.39 % vs. 28.46 %). The difference of 0.75 percentage points is therefore caused by the different test data and not by more training data, even though the rank test is significant ($p = 0.008$). The small configuration (1000/200) is kept for the sweeps because it is about ten times faster and corresponds to the data volume of a personalization session.

4.2.6 Split Type

Table 4.9: Split type. Mean test error over five runs with 98 % CI and the constant baseline of the respective test subset.

Split	Mean test error (%)	98 % CI (%)	p vs. †	Constant baseline (%)
`norm_random`	27.72	[27.64, 27.80]	0.008	27.66
`norm_subject`†	28.39	[28.26, 28.53]	—	28.46

The random split gives the lower error (Table 4.9). Again, most of the difference comes from the test subsets: the constant baseline is already 0.80 percentage points lower on the `norm_random` subset, and both models are within 0.07 percentage points of their own baseline. In addition, the random split lets the same persons appear in training and test data, which makes the task easier and the estimate optimistic. The subject-disjoint split `norm_subject` is therefore used as the default: it is the only one that measures generalization to unseen persons, which is the situation before personalization.

4.3 Spatial Error Analysis and Loss Functions

The mean error of Section 4.2 hides *where* on the screen the errors occur. This section first analyses the spatial error distribution (Sections 4.3.1 and 4.3.2). The observed pattern motivated position-weighted loss functions (Section 4.3.3), which are evaluated in Section 4.3.4.

4.3.1 Screen Partition

The normalized screen $[0,1]^2$ is described by a 3×3 reference grid with three regions:

$$\begin{aligned} \text{Centre:} &\quad (0.50,\, 0.50), \\ \text{Edges:} &\quad (0.10,\, 0.50),\ (0.50,\, 0.10),\ (0.90,\, 0.50),\ (0.50,\, 0.90), \\ \text{Corners:} &\quad (0.10,\, 0.10),\ (0.10,\, 0.90),\ (0.90,\, 0.10),\ (0.90,\, 0.90). \end{aligned}$$

A test sample with target $\boldsymbol{p}$ belongs to grid point $\boldsymbol{g}$ if $\|\boldsymbol{p} - \boldsymbol{g}\|_2 \leq r$ with $r = 0.15$. The error of a grid point is the mean Euclidean error $\|\hat{\boldsymbol{p}} - \boldsymbol{p}\|_2$ of its samples (in normalized units, i.e. a fraction of the screen side; the diagonal is $\sqrt{2} \approx 1.41$). The error of a region is the mean over all samples of its grid points. These definitions are used in all figures and tables of this section.

4.3.2 Error Heat Maps

Figure 4.3 uses the 200 test predictions of the default model (`norm_subject`, L1 loss; file `results/heatmap/base_loss1.csv`). For a cursor position $\boldsymbol{c}$ (centre, edge or corner), the mean error $\bar{e}(\boldsymbol{c})$ of all samples within radius r is computed and shown as an isotropic Gaussian

$$H(x,y) = \bar{e}(\boldsymbol{c}) \exp\left(-\frac{\|(x,y) - \boldsymbol{c}\|_2^2}{2\sigma^2}\right), \qquad \sigma = 0.20. \tag{4.3}$$

The colour therefore encodes the *size* of the local mean error, and the position of the bright spot shows *where* it was measured. All three panels use the same colour scale (0 to the largest value), so the colours can be compared directly.

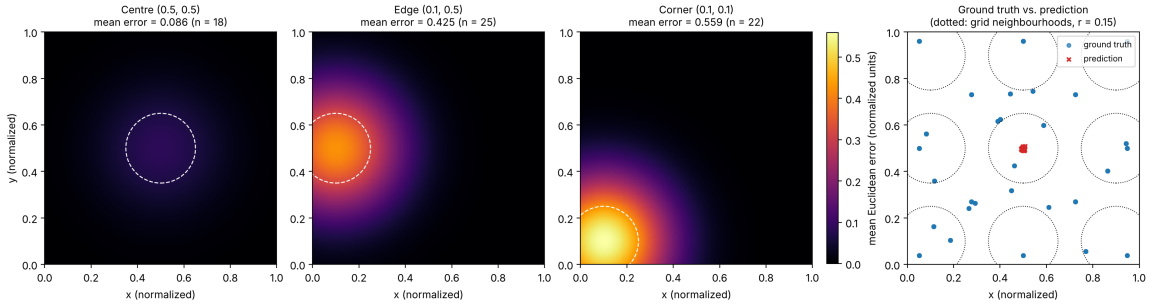


Figure 4.3: Spatial error of the default model (L1 loss, `norm_subject`, 200 test samples). Panels 1–3: Gaussian error heat maps ($r = 0.15$, $\sigma = 0.20$, common colour scale) at the centre $(0.5, 0.5)$, the left edge $(0.1, 0.5)$ and the lower left corner $(0.1, 0.1)$. The error grows from the centre (0.086) to the edge (0.425) and the corner (0.559). Panel 4: targets (blue) and predictions (red); all predictions lie close to the screen centre.

The error increases monotonically from the inside to the outside: the mean error is 0.086 in the centre region ($n = 18$), 0.432 at the edges ($n = 62$) and 0.586 in the corners ($n = 42$).

Edge errors are about five times and corner errors about seven times larger than centre errors.

The fourth panel explains this pattern. All predictions lie in a very small area around $(0.5, 0.5)$: their standard deviation is about 0.004 in both coordinates, while the targets have a standard deviation of about 0.28. The model therefore predicts almost the same point for every image, and its error is simply the distance of the target to the screen centre. A constant centre predictor gives practically the same values (centre 0.085, edges 0.432, corners 0.587; overall $E_{\text{diag}} = 28.45\,\%$ versus $28.47\,\%$ for the model).

4.3.3 Loss Functions

The high edge and corner errors motivated the idea to give these regions more weight in the loss. Let $\hat{\boldsymbol{p}}_i = (\hat{x}_i, \hat{y}_i)$ be the prediction and $\boldsymbol{p}_i = (x_i, y_i)$ the target of sample i in a batch of size N.

L1 loss (`nn.L1Loss`, "standard").

$$\mathcal{L}_1 = \frac{1}{2N} \sum_{i=1}^{N} \big(|\hat{x}_i - x_i| + |\hat{y}_i - y_i| \big). \tag{4.4}$$

The gradient has a constant magnitude independent of the error size. The loss is therefore robust against outliers (for example wrong labels during target transitions), but it treats all screen positions equally.

Smooth L1 loss (`nn.SmoothL1Loss`, $\beta = 1$). For a residual u:

$$\ell_\beta(u) = \begin{cases} \dfrac{u^2}{2\beta}, & |u| < \beta, \\ |u| - \dfrac{\beta}{2}, & \text{otherwise.} \end{cases} \tag{4.5}$$

It is quadratic for small and linear for large residuals. Because all residuals in $[0, 1]^2$ are smaller than $\beta = 1$, it behaves here like a scaled squared error: the gradient becomes small when the error is small.

Sensitive L1 loss (position-weighted, `sensitive`). Each sample is weighted by its L_1 distance from the screen centre:

$$d_i = |x_i - 0.5| + |y_i - 0.5| \in [0, 1], \qquad w_i = 1 + d_i \in [1, 2], \tag{4.6}$$

$$\mathcal{L}_{\text{sens}} = \frac{1}{N} \sum_{i=1}^{N} w_i \cdot \frac{|\hat{x}_i - x_i| + |\hat{y}_i - y_i|}{2}. \tag{4.7}$$

A corner sample has twice the weight of a centre sample, an edge sample about 1.4 times. This increases the gradient for outer targets and should pull the predictions away from the centre. (The earlier draft defined the weight as $w = 1/d$; this does not match the implementation in `model.py` and would be infinite at the centre.)

The "improved sensitive loss" in Table 4.11 was a further variant from an exploratory notebook. Its exact formula is not contained in the current code, so it is reported only for completeness.

4.3.4 Results of the Loss Comparison

Table 4.10 combines the global comparison from the five-run sweep with the spatial errors of the regions defined in Section 4.3.1. It replaces the two separate tables of the earlier draft (global loss table and 3×3 grid table).

Table 4.10: Comparison of L1 and Sensitive L1 loss. Left: mean test error E_{diag} over five runs (`exp.py`, 98 % CI) and mean run time. Right: mean Euclidean error per screen region (normalized units; one earlier run each, 200 test samples, $r = 0.15$; n = number of samples). Last row: constant screen-centre predictor on the same 200 test samples.

	Global (5 runs)			Spatial error (1 run)		
Loss	E_{diag} (%)	98 % CI (%)	Run time (min)	Centre ($n = 18$)	Edges ($n = 62$)	Corners ($n = 42$)
L1 (standard)[†]	**28.32**	[28.23, 28.42]	0.011	0.086	0.432	0.586
Sensitive L1	28.48	[28.19, 28.78]	0.021	0.090	0.431	0.585
Constant centre	28.45[a]	—	—	0.085	0.432	0.587

[a] Evaluated on the same 200 test samples as the spatial analysis.

Table 4.11: Auxiliary experiment: spatial error of four loss functions after 15 epochs on a training set with 52 corner samples (two runs, A and B, values as logged). These numbers come from an exploratory notebook; the computation of the region values of this run is not contained in the current code, so they are not directly comparable with Table 4.10.

	Total (%)		Centre (%)		Edges (%)		Corners (%)	
Loss	A	B	A	B	A	B	A	B
L1 (`nn.L1Loss`)	24.54	24.63	17.01	16.74	28.28	28.98	34.93	35.07
Smooth L1 (`nn.SmoothL1Loss`)	23.94	24.91	15.40	16.41	29.78	30.59	34.13	35.17
Sensitive L1	25.57	25.55	19.11	18.61	29.87	29.93	33.41	34.20
Improved sensitive L1	25.53	27.75	19.20	21.86	29.17	35.03	33.77	31.58

Observations.

- Globally, L1 is slightly but significantly better than the sensitive loss: 28.32 % versus 28.48 % (Mann–Whitney $p = 0.032$, Table 4.10); the L1 value is therefore printed in bold. The sensitive loss also varies more between runs. Both values lie close to the constant baseline (28.46 %).
- Spatially, the position weighting changes the region errors by less than 0.005 normalized units. The edge and corner errors stay about five to seven times higher than the centre error.
- Both models have the same spatial profile as a constant centre predictor. The weighting increases the gradient for outer samples, but this is not enough to make the model leave the centre solution.
- In the auxiliary experiment (Table 4.11) the sensitive variants slightly reduce the corner error (Sensitive L1 by 0.9–1.5, the improved variant by 1.2–3.5 percentage points), but they increase the centre error by 1.9–5.1 percentage points and the total error in all runs. The two runs differ considerably, so these trends are not reliable.

Conclusion. The spatial analysis showed *what* the problem is: the errors are concentrated at the edges and in the corners. The position-weighted loss was the attempt to reduce these errors. It did not improve the corners and was globally even slightly but significantly worse than L1 ($p = 0.032$); in the auxiliary experiment it increased the centre error. The standard L1 loss is therefore kept. It gave the lower error, it is simpler, it has no additional hyperparameter, and its constant gradient magnitude makes it robust against label noise in the personal recordings. The analysis also shows *why* the weighting could not help: the large outer errors are not caused by a wrong weighting of screen regions but by the fact that the model barely leaves the screen centre (Section 4.8).

4.4 Personalization Sessions

4.4.1 Setup

In each personal session the head of the GazeCapture model is trained again on the images of one recording (FC only, Sigmoid, L1, $\eta = 10^{-4}$, batch size 32, AdamW). Each session is split randomly into 70 % training, 15 % validation and 15 % test frames (seed 42). Early stopping (patience 3) uses the validation error, and the test error is reported at the epoch with the lowest validation error. If the best epoch is 0, the personalization did not improve the validation error and the reported test error is that of the unchanged base model.

Of the 27 recorded sessions, 18 are used: Gen 1 (01–04), Gen 2 (05–09) and Gen 4 (19–27). Gen 3 (10–18) was discarded because it showed no learning effect. Table 4.12 lists the recording conditions. All values are taken from the notebook logs of `personal_compare.ipynb`; the validation and test errors were computed with the old evaluation function and are multiplied by $\sqrt{2}$ (Section 4.1.3). The constant and random baselines were computed with the correct formula on all frames of a session. The random seeds of these runs were not fixed, so a repeated run gives a somewhat different result (Section 4.4.4). The new script `personal_exp.py` fixes all seeds and keeps the backbone, including its BatchNorm statistics, completely frozen (in the old notebook the BatchNorm statistics were updated during training).

Table 4.12: Recording conditions of the 18 sessions. Gen 1: continuous target movement without jumps; Gen 2: target with random jumps; Gen 4: 5-s segments (0.5 s WAIT at a new start point, 4.5 s MOVE), 90 % of the start points in the screen corners. Duration and operating system were only varied in Gen 4 (Gen 1/2: Ubuntu, duration not logged).

Sessions	Generation	Glasses	Light / background
01, 03, 04	Gen 1	with	01, 04: bright; 03: dark
02	Gen 1	without	bright
05, 07, 08	Gen 2	with	05: bright; 07: dark; 08: very dark
06, 09	Gen 2	without	06: bright; 09: very dark
Gen 4	(duration / OS)		
19, 20	Gen 4	with	clear; 120 s / 90 s; Ubuntu
21, 22	Gen 4	without	clear; 120 s / 90 s; Ubuntu
23 / 24	Gen 4	with / without	noisy; 120 s; Ubuntu
25 / 26	Gen 4	with / without	noisy; 120 s; Windows
27	Gen 4	with	noisy; 90 s; Windows

4.4.2 Results of All Sessions

Table 4.13 shows every session with both GazeCapture base models. For sessions 19–27 only the `norm_subject` base model has been run so far; the missing `norm_random` runs are part of Experiment E1 (Section 4.7).

Personalization works. In 16 of 18 sessions the personalized model is clearly better than the constant baseline (Table 4.13 and Figure 4.4). Over all sessions the test error falls from 29.85 % to 19.85 %, a reduction of one third (Wilcoxon signed-rank test $p < 10^{-4}$; paired t-test $p < 10^{-6}$). The best session, S02 (Gen 1, bright room, no glasses), reaches 12.04 %. The GazeCapture base model alone is close to the constant baseline (Section 4.8), so almost the whole gain comes from the few minutes of personal data.

Table 4.13: Personalization of the 18 sessions with two base models (original labels, random 70/15/15 split). Val.: best validation error; Test: test error at the best validation epoch; t: training time of the session run in minutes (see Section 4.4.3); Const.: constant baseline of the session. Errors in % (Equation (4.1)). Group rows: mean and 98 % CI over the sessions of the group; —: not run yet.

		Base model `norm_subject`			Base model `norm_random`			
Session	Gen.	Val. (%)	Test (%)	t (min)	Val. (%)	Test (%)	t (min)	Const. (%)
01	1	15.16	16.16	13.3	12.91	14.48	25.7	28.78
02	1	12.88	12.04	10.3	13.40	12.92	7.8	29.30
03	1	16.41	17.11	14.9	20.63	21.93	6.9	28.76
04	1	17.54	15.33	10.4	17.58	15.76	8.6	28.51
05	2	24.20	18.07	7.5	26.29	22.03	3.9	30.88
06	2	18.65	14.83	9.0	19.15	15.67	6.4	27.45
07	2	24.70	21.83	5.5	26.27	23.00	3.4	28.46
08	2	26.58	27.58	1.0	26.35	28.53	1.0	26.95
09	2	27.33	26.18	1.9	23.94	21.20	3.5	25.87
19	4	19.82	17.65	21.2	—	—	—	30.79
20	4	21.15	22.87	9.7	—	—	—	31.82
21	4	12.24	13.22	12.2	—	—	—	31.62
22	4	16.76	16.69	11.7	—	—	—	30.57
23	4	21.33	21.48	15.5	—	—	—	31.81
24	4	28.48	23.25	8.4	—	—	—	33.09
25	4	31.24	27.07	6.1	—	—	—	31.25
26	4	21.33	19.52	16.6	—	—	—	29.83
27	4	26.76	26.38	7.2	—	—	—	31.52
Gen 1 (4)		15.50	15.16	12.2	16.13	16.27	12.2	28.84
		[10.96, 20.03]	[10.16, 20.16]	[7.1, 17.4]	[7.83, 24.43]	[7.32, 25.23]	[0.0, 32.6][a]	[28.08, 29.59]
Gen 2 (5)		24.29	21.70	5.0	24.40	22.09	3.6	27.92
		[18.58, 30.00]	[12.71, 30.68]	[0.0, 10.8][a]	[19.19, 29.61]	[14.39, 29.78]	[0.4, 6.9]	[24.74, 31.10]
Gen 4 (9)		22.12	20.90	12.1	—	—	—	31.37
		[16.42, 27.82]	[16.49, 25.31]	[7.3, 16.8]				[30.48, 32.26]
All (18)		**21.25**	**19.85**	**10.1**				**29.85**
		[17.90, 24.61]	[16.87, 22.83]	[7.1, 13.2]				[28.67, 31.03]
01–09 (9)		20.39	18.79	8.2	20.72	19.50	7.5	28.33
		[15.20, 25.57]	[13.68, 23.91]	[3.6, 12.8]	[15.54, 25.91]	[14.60, 24.40]	[0.5, 14.5]	[26.93, 29.72]

[a] Lower limit of the t interval below zero (few sessions, large spread), truncated at 0.

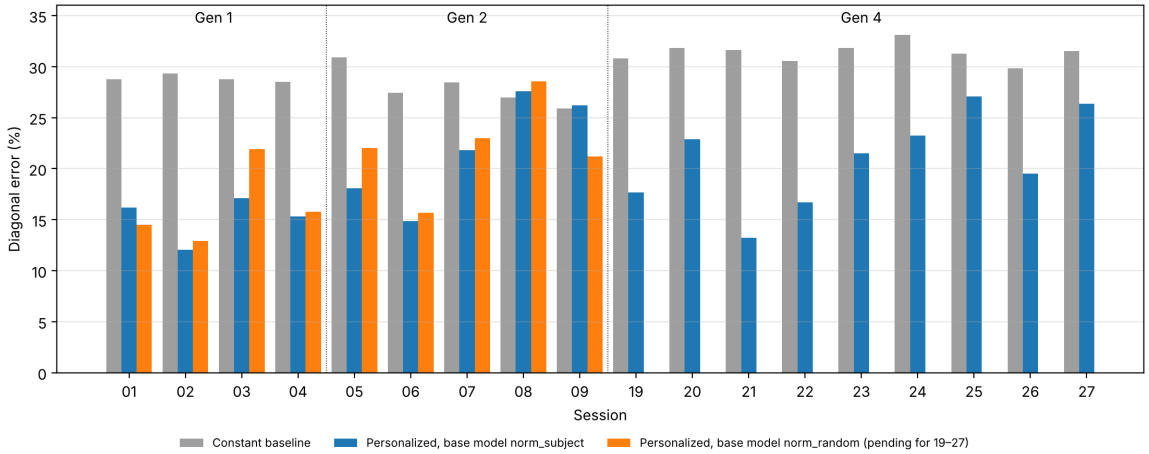


Figure 4.4: Test error of the 18 sessions after personalization with the two base models, compared with the constant baseline of each session (data of Table 4.13).

Groups. The reduction compared with the constant baseline is about 47 % in Gen 1, 21 % in Gen 2 and 33 % in Gen 4. In Gen 4 the improvement over the constant baseline is significant (Wilcoxon $p = 0.004$); for the smaller groups Gen 1 ($n = 4$) and Gen 2 ($n = 5$) the test cannot reach $p < 0.05$ although the effect is large in Gen 1. The differences between the generations are not significant (Kruskal–Wallis $p = 0.08$) and must not be read as an effect of the recording protocol alone (Section 4.6).

When personalization fails. Sessions 08 and 09 were recorded in a very dark room. Here the test error is equal to or slightly above the constant baseline. In S08 the best epoch is 0: no training step improved the validation error, so the reported model is the unchanged base model. Good illumination is therefore a precondition for webcam personalization.

Base model. For sessions 01–09 both base models give the same result: 18.79 % with `norm_subject` and 19.50 % with `norm_random` (Wilcoxon $p = 0.43$). This is consistent with the main finding that the base model mainly provides an initialization and that the personal data determine the result. The `norm_subject` base model is used for the following experiments, because it was trained without information about the test persons.

Validation and test error. In most sessions the validation error is close to the test error. Large gaps (S05, S24, S25) occur when the validation split happens to contain more difficult frames; with only 15 % of a short recording, both splits are small.

4.4.3 Training Time of a Session

The time t in Table 4.13 is the wall-clock duration of *one complete personalization run* of a session: loading the base model, the evaluation of the unchanged model (epoch 0), all training epochs until early stopping and the evaluation of the training and validation data after every epoch. It does not include the final test evaluation. It is therefore neither a time per epoch nor an average over several runs. It depends on the number of frames of the session and on the number of epochs before early stopping: Gen 2 recordings are short (about 30 s, 150–215 frames) and need about 5 min, Gen 1 and Gen 4 recordings (75–120 s, 445–720 frames) about 12 min. In contrast, the "Run time" of the hyperparameter tables (Section 4.1.4) is the mean over five repeated runs of a GazeCapture configuration.

4.4.4 Run-to-Run Variability and Label Cleaning (Gen 4)

In Gen 4 every segment starts with a 0.5 s WAIT phase in which the target appears at a new position and the user still has to find it. These frames (10 % of the labels) can have a wrong gaze label. They are logged exactly and can be removed.

Correction. The earlier draft compared Gen 4 with "original" and "cleaned" labels and reported a mean gain of 0.93 percentage points. A check of the data shows that this comparison is not valid: the files `norm_labels_clean.csv` used for these runs were byte-identical to `norm_labels.csv` (the notebook log shows the same number of frames, e.g. 574, 540 and 720), so no WAIT frame was removed. Correct cleaned files have now been generated from the logged phases (`session_design.py --write-clean`; 10.0–10.5 % of the labels removed per session); the comparison has to be repeated (Experiment E1b).

The invalid comparison has one useful result: it consists of two runs of the same configuration on *identical* data. Table 4.14 therefore shows the run-to-run variability of a personalization run that comes only from random initialization, data order and augmentation.

Table 4.14: Two runs (A, B) of the same personalization on identical data (Gen 4, base model `norm_subject`). Test error at the best validation epoch (%); Δ = B − A.

Session	19	20	21	22	23	24	25	26	27	Mean $\lvert\Delta\rvert$
Run A	17.65	22.87	13.22	16.69	21.48	23.25	27.07	19.52	26.38	
Run B	16.67	17.39	13.22	17.64	24.14	23.80	22.13	19.15	25.57	
Δ	−0.98	−5.48	0.00	+0.95	+2.65	+0.55	−4.94	−0.37	−0.80	1.86

Two runs of the same session differ on average by 1.9 and at most by 5.5 percentage points (standard deviation of the difference 2.7 percentage points). Differences between *single* sessions or *single* runs of this size are therefore not interpretable. For this reason the new

runs fix all random seeds and the data loading comparison (Section 4.5) uses three seeds per configuration.

4.4.5 Recording Conditions (Gen 4)

Gen 4 allows comparisons in which only one factor changes. Each value is the difference of two single sessions (Run A, percentage points; positive = the first condition is worse):

- **Glasses** (with − without): +4.4 (S19–S21), +6.2 (S20–S22), −1.8 (S23–S24), +7.6 (S25–S26);
- **Duration** (90 s − 120 s): +5.2 (S20–S19), +3.5 (S22–S21);
- **Background** (noisy − clear): +3.8 (S23–S19), +10.0 (S24–S21);
- **Operating system** (Windows − Ubuntu): +5.6 (S25–S23), −3.7 (S26–S24).

The tendency is plausible: glasses, a shorter recording and a noisy background increase the error. However, most of these differences are of the same size as the run-to-run variability (Table 4.14), and each condition is represented by one session. They are therefore descriptive and not statistically tested; only the background effect in S24–S21 (10.0 percentage points) is clearly larger than the variability.

4.5 Data Loading Strategies

The random split of Section 4.4 has a weakness: neighbouring frames of a video are almost identical (6 labels per second), so the test set contains near-duplicates of training frames. This is a form of data leakage that makes the random split optimistic. The following comparison measures how large this effect is.

Sessions. Sessions 21 and 22 are used. They are selected by a fixed rule: they are the only two sessions that differ in a single factor (duration, 120 s vs. 90 s) and were otherwise recorded under the best conditions (Gen 4, no glasses, clear background, Ubuntu, same day). This makes them suitable for the two-video transfer, and both have enough frames (720 and 540) for temporal blocks.

Base models. The two base models are retrained with the selected configuration of Section 4.2 (FC only, Sigmoid, L1, $\eta = 10^{-4}$, batch 32, 1000/200 random frames, seed 0, validation-based early stopping) for `norm_subject` and `norm_random`. Of five seeds the run with the lowest *validation* error is kept and saved (`train_base.py`, `models/base/base_<split>.pt` with a JSON file that documents the configuration and all seeds). The comparison uses the `norm_subject` base model.

Strategies.

Random sampling (`--split random`): 70 % / 15 % / 15 % of the frames are drawn at random from the session (reference; contains near-duplicates).

Sequential sampling (`--split sequential`): the frames are kept in recording order; the first 70 % are used for training, the next 15 % for validation and the last 15 % for testing. One second (6 labels) at the start of the validation and of the test block is dropped, so that no neighbouring frames lie on both sides of a block border. This corresponds to the real use case: the user calibrates first and uses the tracker afterwards.

Two-video training (`--split two_videos`): 80 % of the frames of the first session are used for training and the remaining 20 % (random) for validation; the second session is used *only* for testing (all frames). Both directions are run (21→22 and 22→21). This measures whether a calibration transfers to a new recording of the same person.

Fair comparison. All strategies use the same base model, the same training settings (head on cached backbone features, Section 4.7), early stopping on their own validation data and the same three seeds (42, 43, 44). For every run the constant baseline (mean of the *training* targets, evaluated on the test frames of that run) and the error of the unchanged base model on the same test frames are reported, so that the different test sets can be compared. No test frame is used for training or for model selection.

Results pending (Experiment E2). Expected outcome: the sequential and the two-video errors are higher than the random-split errors; the difference shows how optimistic the random split is. The table will report, per session and strategy, the mean test error over the three seeds, the constant baseline and the base-model error.

4.6 Personalization Session Design

The session design determines which screen positions are covered by the calibration data. Because the errors are largest at the edges and in the corners (Section 4.3), the design aims to cover these regions well. Three designs were recorded:

- **Standard video** (Gen 1, `base_Code.py`): the target moves continuously over the screen, without jumps;
- **Random jumping points** (Gen 2): the target jumps every 1.4–2.7 s to a uniformly drawn new start point;
- **Corner-weighted jumping points** (Gen 4, `base_Code_rand.py`): 5-s segments with 0.5 s WAIT at a new start point and 4.5 s MOVE; 90 % of the start points lie in the four corner regions (each 10 % × 10 % of the screen).

4.6.1 Spatial Coverage and Transition Samples

Table 4.15 is computed directly from the label and position files of the 18 sessions (`session_design.py`); no training is needed. The region shares use the screen partition of Section 4.3.1 ($r = 0.15$). Transition samples are labels recorded while the gaze is still moving to a new target: in Gen 4 the logged WAIT phase, in Gen 2 the labels within 0.5 s after a jump (estimated, no phase logging), in Gen 1 none.

Table 4.15: Spatial coverage and transition samples per design (mean over the sessions of each generation). Cells: share of the 100 cells of a 10×10 screen grid that contain at least one label; Dist.: mean distance of the targets from the screen centre (normalized units).

Design	Labels	Length	Jumps	Transition	Share of labels (%)			Cells	Dist.
		(s)		(%)	Centre	Edges	Corners	(%)	
Standard (Gen 1, 4 sess.)	493	82	0	0	7.1	25.0	23.0	97	0.383
Random jumps (Gen 2, 5 sess.)	179	30	16	28.5[a]	6.3	23.0	26.3	57	0.384
Corner-weighted (Gen 4, 9 sess.)	644	107	20	10.1	6.1	19.5	36.1	91	0.420

[a] Estimated: labels within 0.5 s after a jump.

Observations.

- **Corner weighting works as intended:** Gen 4 has 36 % of its labels in the corner neighbourhoods compared with 23 % (Gen 1) and 26 % (Gen 2), and the largest mean distance from the centre.
- **Continuous movement covers the screen most evenly:** Gen 1 reaches 97 % of the grid cells. The short Gen 2 recordings (30 s) cover only 57 %.
- **Transition samples:** with jumps every 1.4–2.7 seconds and no phase logging, up to a third of the Gen 2 labels fall into a transition; Gen 4 has 10 %, which are logged exactly and can be removed.

4.6.2 Personalization Performance

The mean test errors are 15.16 % (standard, Gen 1), 21.70 % (random jumps, Gen 2) and 20.90 % (corner-weighted, Gen 4; Table 4.13); the differences are not significant (Kruskal–Wallis $p = 0.08$). With the existing data these numbers cannot be attributed to the design alone: (i) the generations also differ in illumination, background, glasses, operating system and recording length (Gen 2: 30 s); (ii) the random split favours the continuous Gen 1 videos, because neighbouring, almost identical frames end up in training and test data; (iii) the run-to-run variability is of the same size as some of the differences (Table 4.14). A fair comparison of the designs therefore requires the sequential split (no near-duplicates between training and test) for all sessions and the region errors (centre, edges, corners)

from the stored test predictions (Experiment E3, Section 4.7). New recordings are not needed for this step.

4.7 Open Experiments

Table 4.16 lists the experiments that are still required for the results of this chapter. They are run in this order in the notebook `notebooks/personal_compare.ipynb`, which also produces all tables of Sections 4.4 to 4.6. Finished runs are stored immediately in `results/personal/personal_runs.jsonl`; an interrupted notebook can simply be run again.

Feature cache. All new personalization runs keep the backbone completely frozen (including the BatchNorm statistics). It therefore maps an image to the same 512 features in every epoch. These features are computed once per session and base model (without augmentation) and stored; afterwards only the regression head is trained. This makes the experiments feasible on a CPU (seconds per run instead of minutes) and does not change the model that is trained. The only difference to the earlier notebook runs is that no image augmentation is applied during personalization.

Table 4.16: Experiments still required (all in `personal_compare.ipynb`).

ID	Purpose	Runs
E0	Base models for `norm_subject` and `norm_random` (`train_base.py`)	2×5
—	Feature cache: backbone features of all sessions for both base models	36 files
E1	All 18 sessions with both base models, fixed seed; completes Table 4.13 and stores the predictions	36
E1b	Gen 4 with correctly cleaned labels (Section 4.4.4)	9
E2	Data loading strategies on sessions 21 and 22, three seeds (Section 4.5)	18
E3	Sequential split for all 18 sessions; region errors per design (Section 4.6)	18

4.8 Discussion

Main finding. On GazeCapture, all configurations of the hyperparameter study reach practically the error of a constant predictor on the same test data: the best FC-only configurations lie between 28.32 % and 28.39 %, the constant baseline is 28.46 %. The spatial analysis confirms that the predictions are concentrated at the screen centre. Consequently the hyperparameter study separates the configurations mainly by *stability*: options that make training less stable (Gaussian, ReLU, no activation, unfreezing backbone layers, the sensitive loss) are worse, but no option clearly leaves the centre solution. The choices in

Section 4.2 are therefore based on stability, convergence, cost and methodological arguments, as stated in each subsection. Single notebook runs on `norm_random` (Figures 4.1b and 4.2) reached about 23.6 % versus 26.8 % for the constant baseline, so a small learning effect exists, but it is not visible in the five-run means.

Possible reasons. The new protocol already excludes two explanations that were plausible for the earlier runs: the training subset now contains 179 subjects instead of five, and the test set is no longer used for model selection; the collapse remains. The most likely remaining causes are: (i) full camera frames are used, in which the eyes cover only a small part of the image, so the frozen ImageNet features carry little gaze information; (ii) 1000 frames from 179 subjects are only about six frames per person, too few to learn a person-independent mapping; unfreezing more layers with so few data makes training unstable instead of better (Section 4.2.3). Face or eye crops as input and larger training sets are the next steps to test.

Consequences for the code. The evaluation metric was corrected, baselines are computed on the same subset as the model, random subsets (`--sample-seed`) and a subject-disjoint validation set (`--val-size`) were added to `exp.py`, and full fine-tuning (`fc_resnet`) can be included with `--include-resnet`.

Consequence for the thesis. The personalization itself is effective: with the same frozen backbone, a head trained on a few hundred images of one person reduces the error by about one third compared with the constant baseline. The GazeCapture base model mainly provides a stable initialization. This supports the central idea of this work, data-efficient personalization of a pretrained model, while it shows that the generic (person-independent) model is not yet a strong baseline.

4.9 Summary of Experimental Results

Table 4.17 summarizes all experiments. "Significant" means $p < 0.05$ in the test given in the respective section. All errors are diagonal errors (Equation (4.1)) in %. MAE and MSE were not logged in these experiments; they are therefore not reported, and the diagonal error (a scaled Euclidean RMSE) is the only accuracy measure that is available and comparable for all experiments.

Table 4.17: Summary of the experimental results and the selected configuration. GazeCapture: `norm_subject`, 1000/200 random frames, five runs, constant baseline 28.46 %; run time = mean duration of one training run (head on cached features). Personalization: mean over the sessions, time = one session run (Section 4.4.3).

Experiment	Selected	Error (%)	Time (min)	Result and reason
Baselines	—	28.46	—	constant baseline on the same test data; random baseline 40.36 (Section 4.1.6)
Target representation	normalized $[0, 1]^2$	—	—	device-independent; metric well-defined (Section 4.1.5)
Batch size	32	28.39	0.019	no significant difference (Section 4.2.1)
Learning rate	10^{-4}	28.37	0.020	no significant difference; best convergence in the curves (Section 4.2.2)
Trainable layers	FC only	28.36	0.015	lowest and most stable error; FC + Layer 4: 9.2 min, full: 15.4 min (Section 4.2.3)
Activation	Sigmoid	**28.36**	0.022	**significantly** better than Gaussian, ReLU, None (Section 4.2.4)
Loss function	L1	**28.32**	0.011	**significantly** better than the sensitive loss (Section 4.3.4)
Data set size	1000/200	28.39	0.018	larger set better only because of a different test set (Section 4.2.5)
Data split	`norm_subject`	28.39	0.020	only split that measures generalization to unseen persons (Section 4.2.6)
Personalization	FC only, 18 sessions	**19.85**	10.1	**significantly** better than the constant baseline (29.85), −33 % (Section 4.4.2)
Base model	`norm_subject`	18.79	8.2	sessions 01–09: same as `norm_random` (19.50, 7.5 min); 19–27 pending, E1 (Section 4.4.2)
Label cleaning	—	—	—	earlier comparison invalid (identical files); repeat with correct files, E1b (Section 4.4.4)
Data loading strategy	pending	—	—	E2 (Section 4.5)
Session design	—	see text	—	standard 15.16, random jumps 21.70, corner-weighted 20.90; not significant and confounded; corner weighting raises the corner share from 23 to 36 %; fair comparison: E3 (Section 4.6)

References

[1] Kyle Krafka, Aditya Khosla, Petr Kellnhofer, Harini Kannan, Suchendra Bhandarkar, Wojciech Matusik, and Antonio Torralba. Eye tracking for everyone. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2176–2184, 2016. doi: 10.1109/CVPR.2016.239.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[3] Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.